



Sistemas Distribuídos

Professor: Rodrigo da Rosa Righi

Contato: rrrighi@unisinovos.br

Agenda

- * Exclusão Mútua Distribuída
 - * Algoritmo com Servidor Centralizado
 - * Algoritmo Baseado em Anel
 - * Algoritmo Baseado em Multicast e Clocks Lógicos
- * Eleições
 - * Algoritmo Baseado em Anel
 - * Algoritmo de Bully

Exclusão Mútua Distribuída

Necessária para prevenir interferência e garantir consistência no acesso a recursos.

Cenário

Atualização de um
arquivo via NFS

Para manter a
consistência, somente
um usuário poderá
atualizá-lo a cada
instante

O editor deve fazer
um lock do arquivo
antes de editá-lo

Exclusão Mútua Distribuída

Você sabe o que é uma **região crítica**?

Região de código que **não** pode sofrer acesso concorrente por um mais processos. Existe tanto em ambientes multicomputador quanto multiprocessador.



Exclusão Mútua Distribuída

Como Avaliar Algoritmos para Exclusão Mútua

Largura de banda
consumida

Proporcional ao número de mensagens
para entrar e sair de uma região crítica

Atraso do cliente

Tempo entre as operações
de entrar e sair

Exclusão Mútua Distribuída

Algoritmo com Servidor Centralizado

Ideia

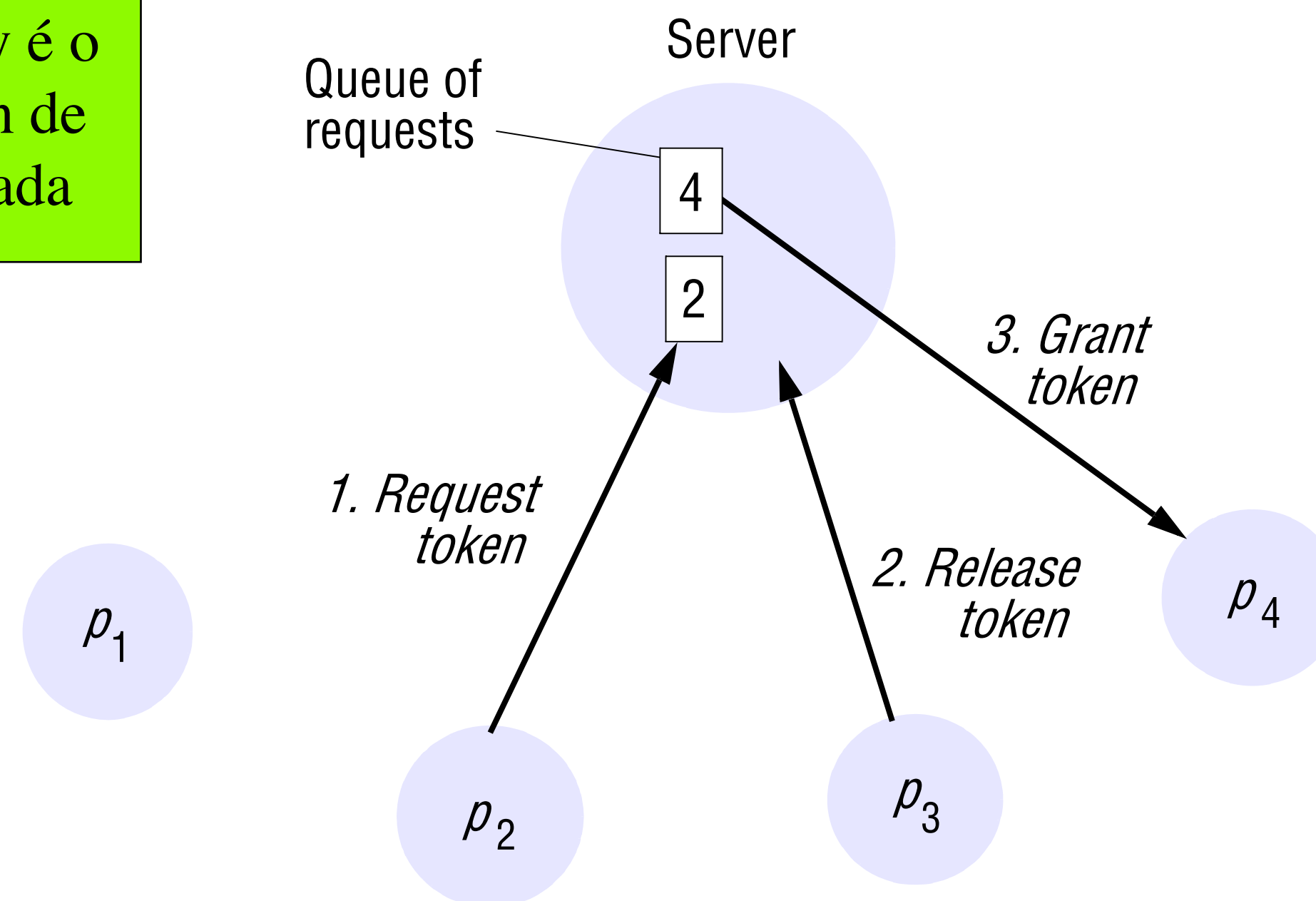
Servidor gerenciando tokens de exclusão mútua para um conjunto de processos

Para entrar na região crítica, um processo envia uma mensagem para o servidor e espera sua permissão

Reply é o token de entrada

Demora vai depender se há ou não processos na fila para serem atendidos

Quando um processo sai da região crítica, ele envia uma mensagem para o servidor



Existe a relação Happened-Before

Exclusão Mútua Distribuída

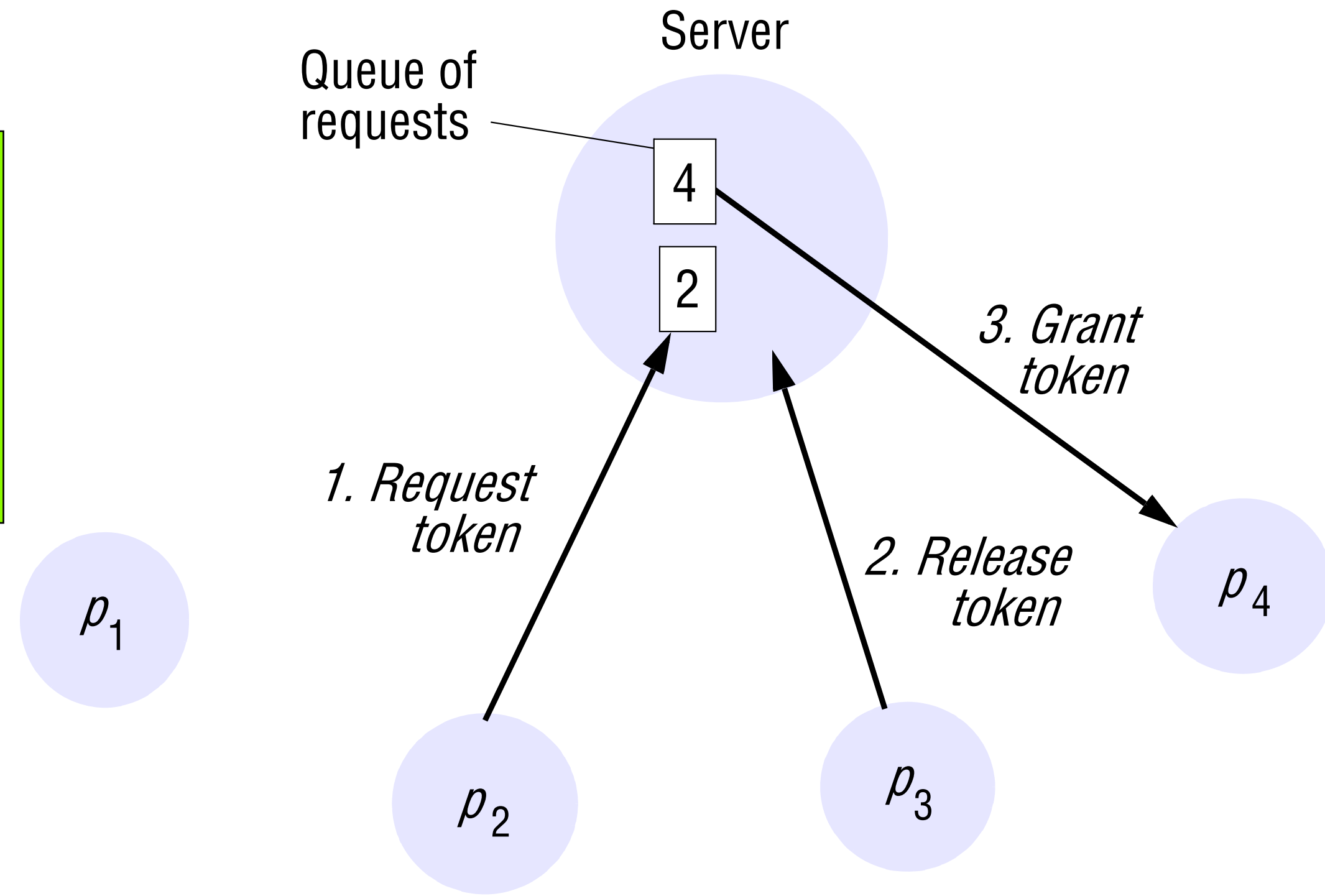
Algoritmo com Servidor Centralizado

Problemas

E se um processo falhar?

Servidor pode se tornar o gargalo do sistema

Necessário um processo a mais para atuar como servidor



Exclusão mútua distribuída

Algoritmo centralizado

- Algoritmo simples de entender e de implementar.
- Requer somente três mensagens para entrar em e sair de região crítica.
- Coordenador central é um ponto único de falha.

Recuperação de uma falha do servidor: um novo servidor deve ser criado ou, então, um dos processos deve assumir esta função. Neste caso, um mecanismo de eleição deve ser executado para definir qual dos processos ativos será o escolhido, já que o servidor tem que ser *único*.

- Outro problema: Como lidar com a falha de um dos processos clientes, principalmente, daquele que detém o *token*?
- Coordenador central pode tornar-se um gargalo!

Exclusão Mútua Distribuída

Algoritmo baseado em um Anel

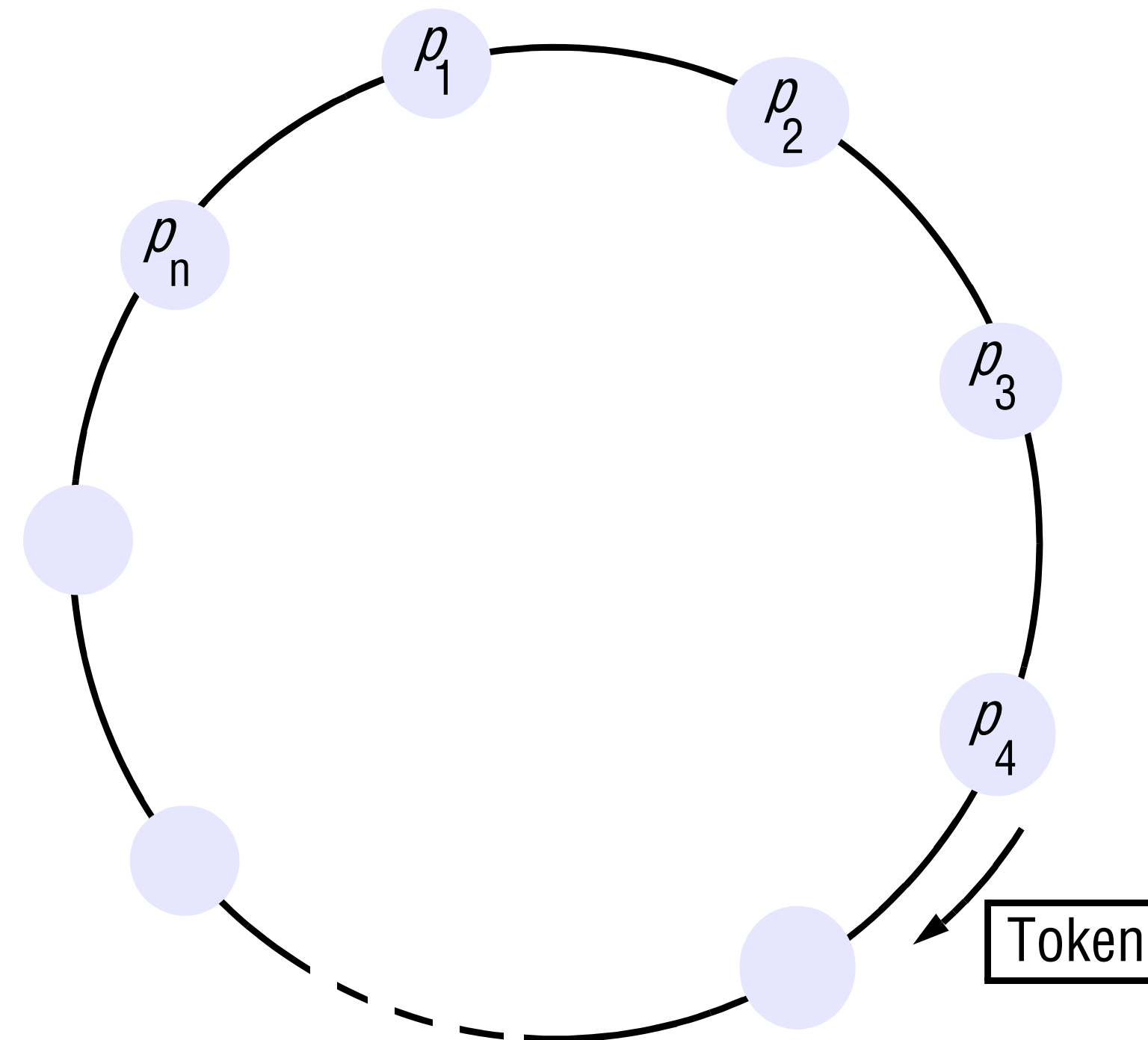
Ideia

Um dos métodos mais simples para implementar exclusão mútua e não há a necessidade de um processo adicional

Processos arranjados num Anel lógico

Cada processo p_i deve ter comunicação com o processo subsequente p_{i+1} (Último processo deve se comunicar com o primeiro)

Token passa pelos processos em uma única direção



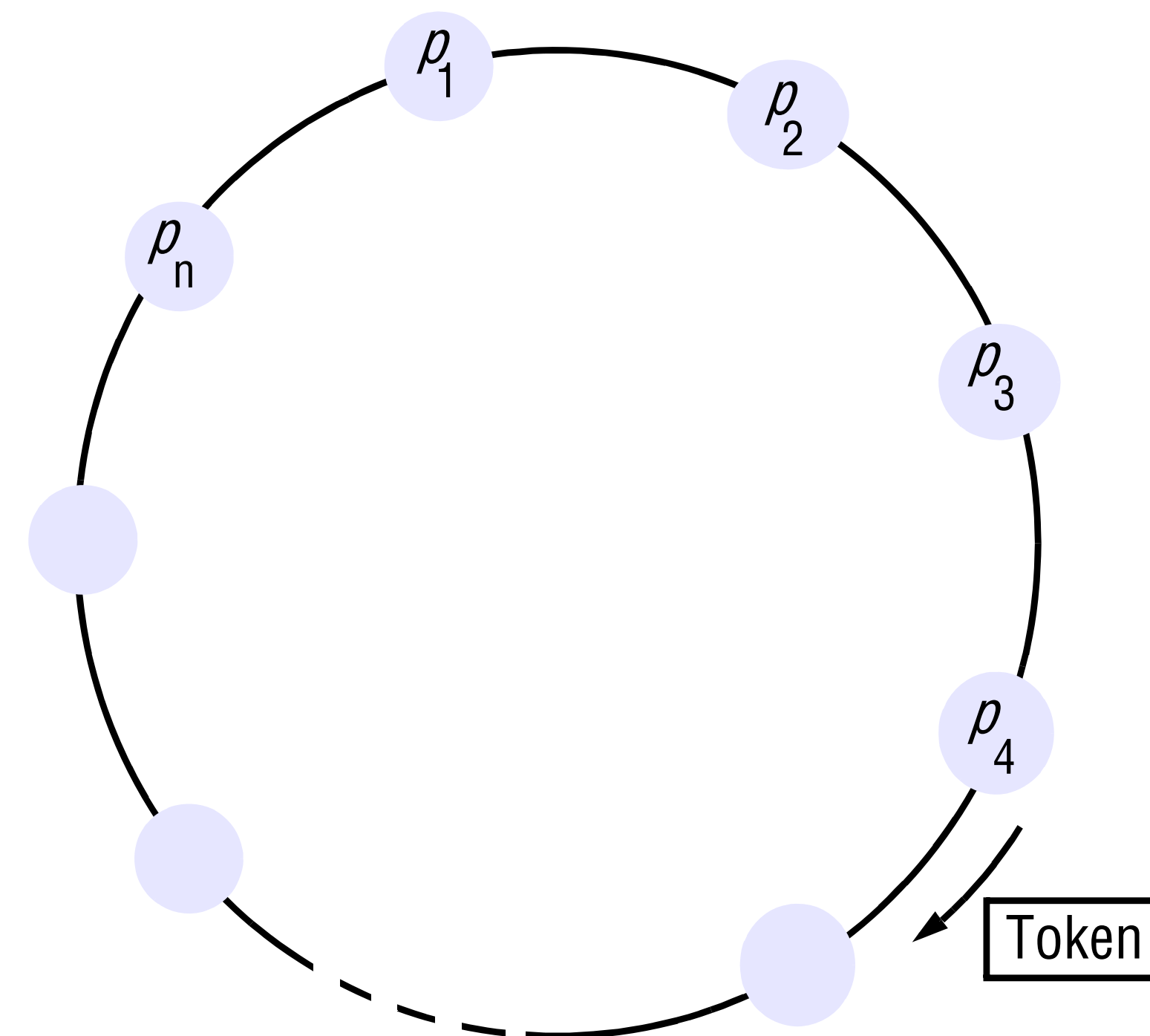
Exclusão Mútua Distribuída

Algoritmo baseado em um Anel

Quero
entrar
na
região
crítica?

Se um processo não quer entrar na região crítica, ele simplesmente repassa o token para o próximo processo

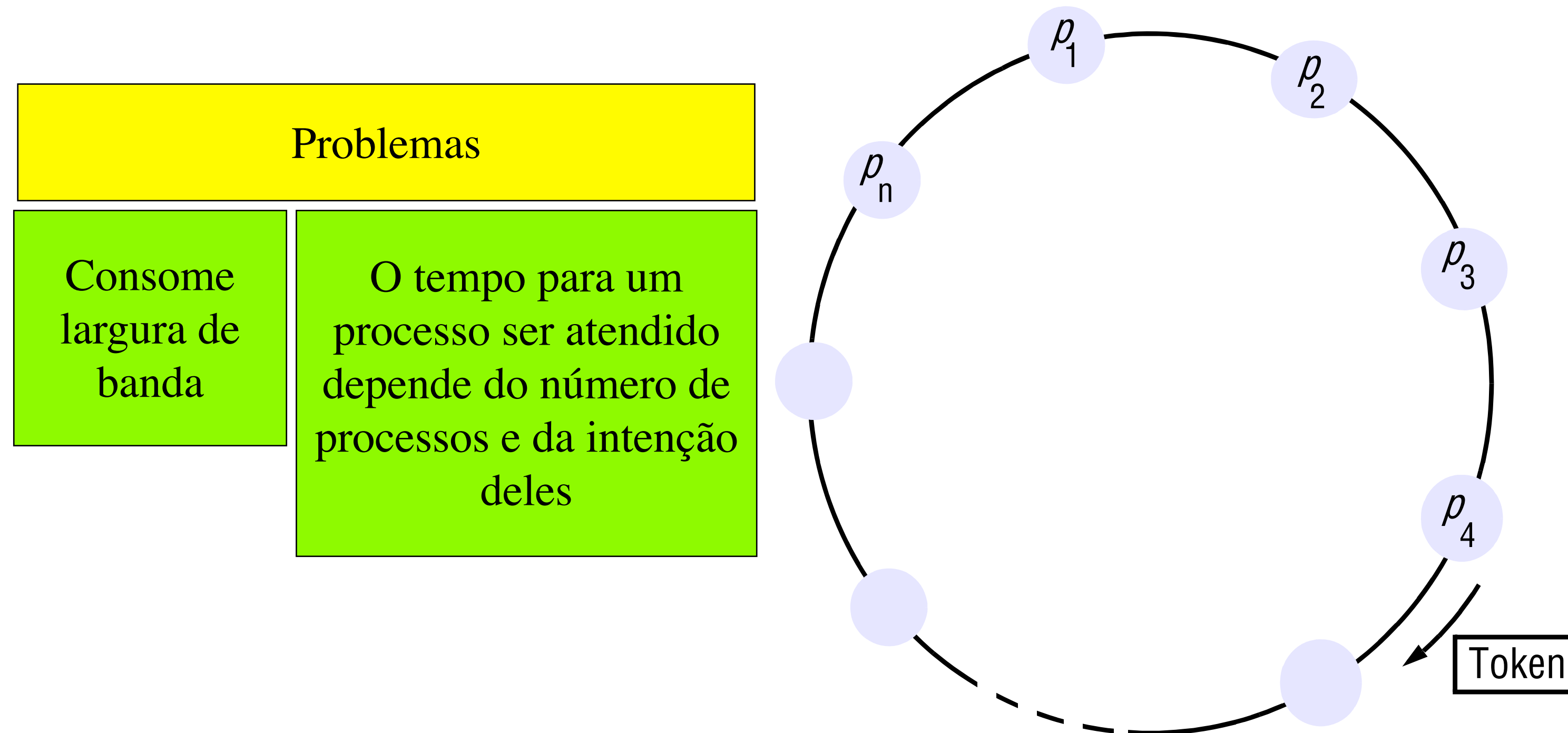
Se um processo quer fazer uso da região crítica, de posse do token ele entra na região e após a saída, repassa o token para o seu vizinho no sentido de rotação do token



Não existe a relação Happened-Before

Exclusão Mútua Distribuída

Algoritmo baseado em um Anel



Exclusão mútua distribuída

Algoritmo em anel (*token-ring*)

- O *token* não é, necessariamente, obtido em ordem “happened-before” (~temporal). Pode levar de 1 a (n-1) mensagens para se obter o *token*, desde o momento em que se torna necessário.
- Mensagens são enviadas no anel mesmo quando nenhum processo requer o *token*.
- Tempo máximo de um ciclo = soma dos tempos de execução das regiões críticas de todos os processos.

Algoritmo em anel (*token-ring*)

- **Token perdido:** recuperação baseada no envio de ACK quando do recebimento do *token*.
- **Processo que falha:** reconfiguração executada para remover o processo do anel. Enquanto isso, a circulação do *token* é interrompida.
- **Se o processo que falha é quem possui o *token*:** um mecanismo de eleição é necessário para escolher um único processo que irá regenerar o *token* e iniciar a sua circulação.

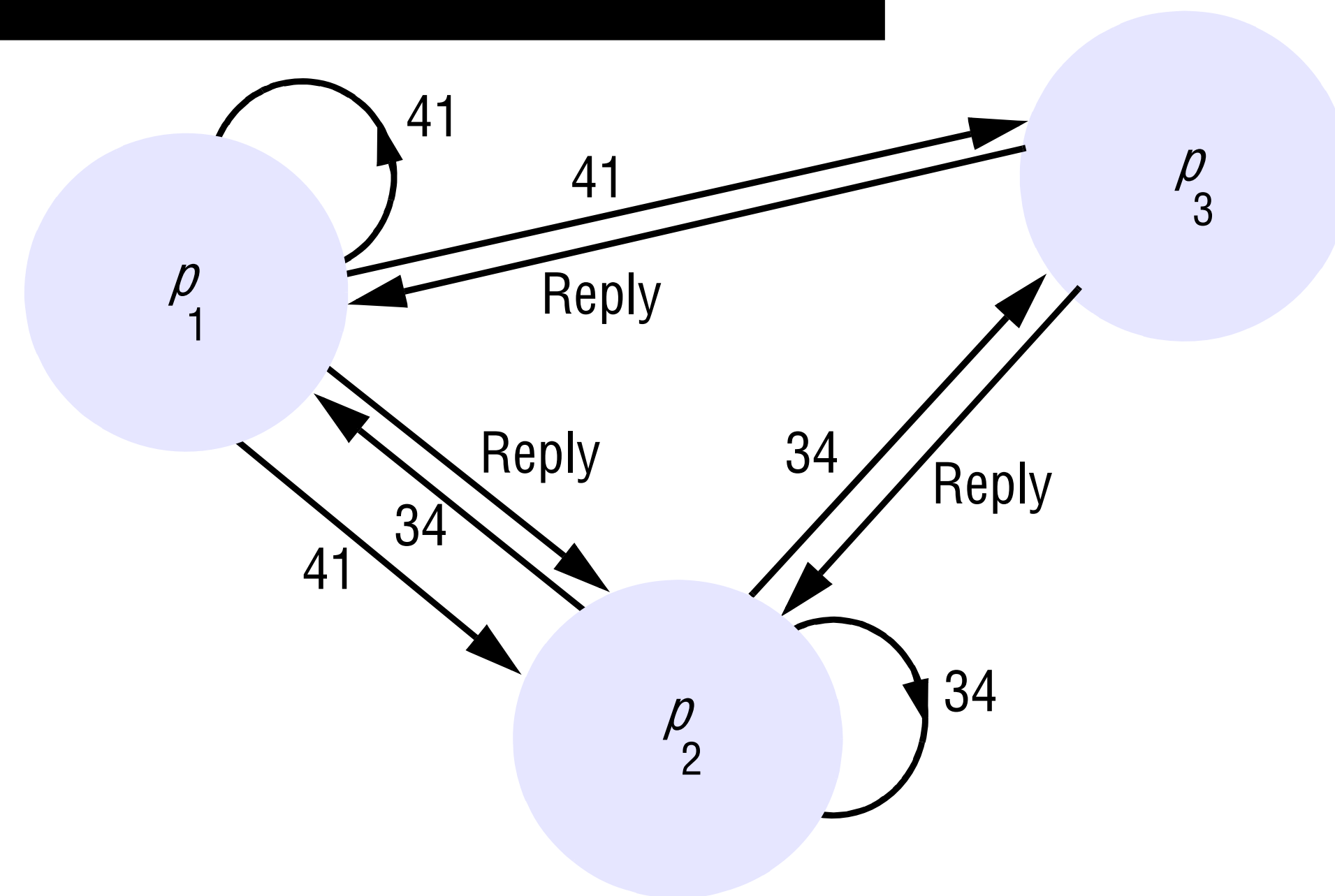
Exclusão Mútua Distribuída

Algoritmo baseado Multicast e Clocks Lógicos

Processos que querem entrar na região crítica devem fazer um multicast de sua intenção

Podem entrar se somente todos os processos fizeram o reply de sua mensagem

Uso de relógio lógico de Lamport



Exclusão Mútua Distribuída

Algoritmo baseado Multicast e Clocks Lógicos

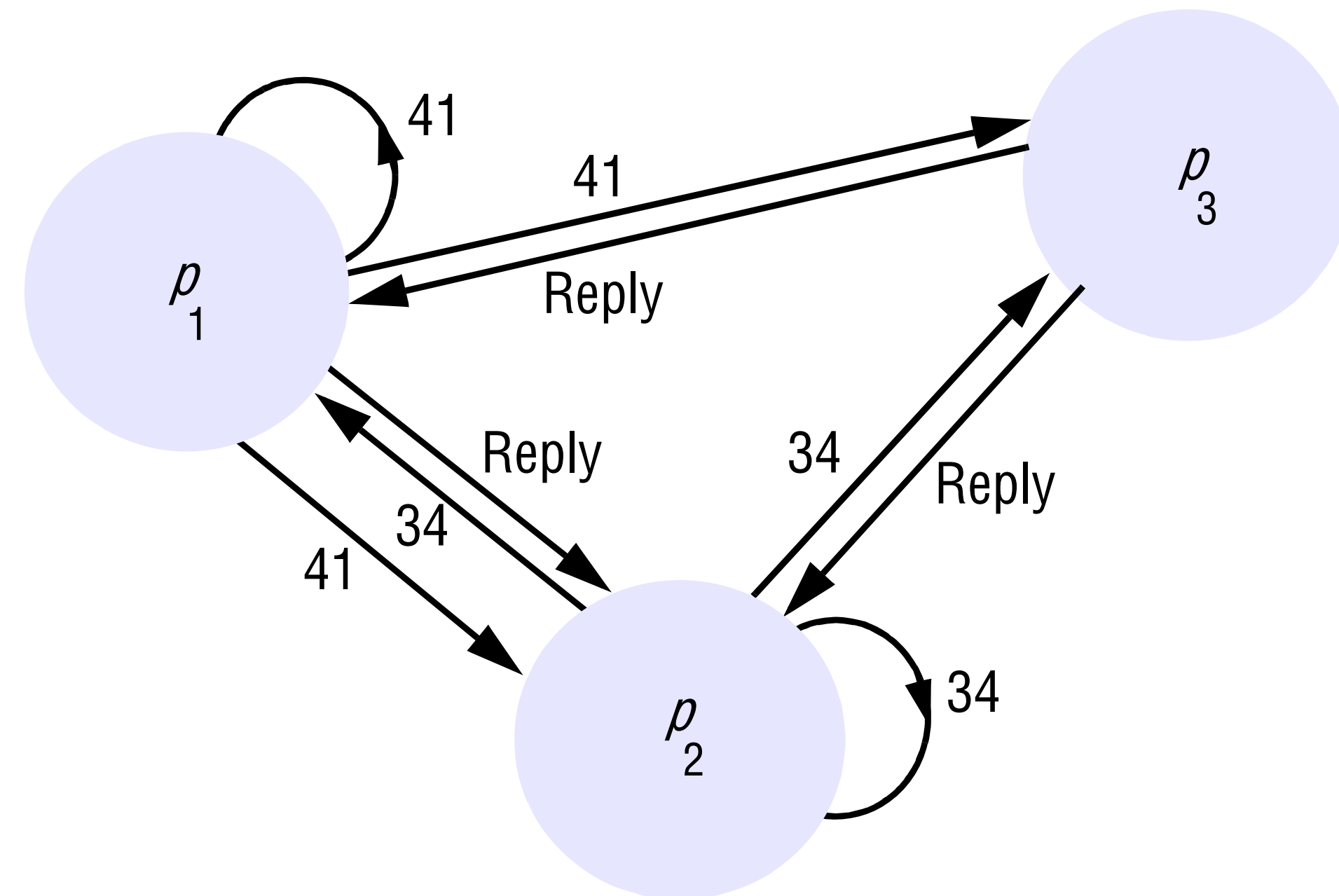
Mensagens de requisição de entrada são da forma $\langle T, P \rangle$

T é um timestamp e P um processo

Cada processo tem noção de seu estado: RELEASED, WANTED e HELD.

Se um processo requisita entrada e se o estado dos demais for RELEASED, eles responderão para o processo que originou a requisição

Se o estado de pelo menos um processo for HELD, ele não irá responder e o requisitante não conseguirá entrar na região crítica

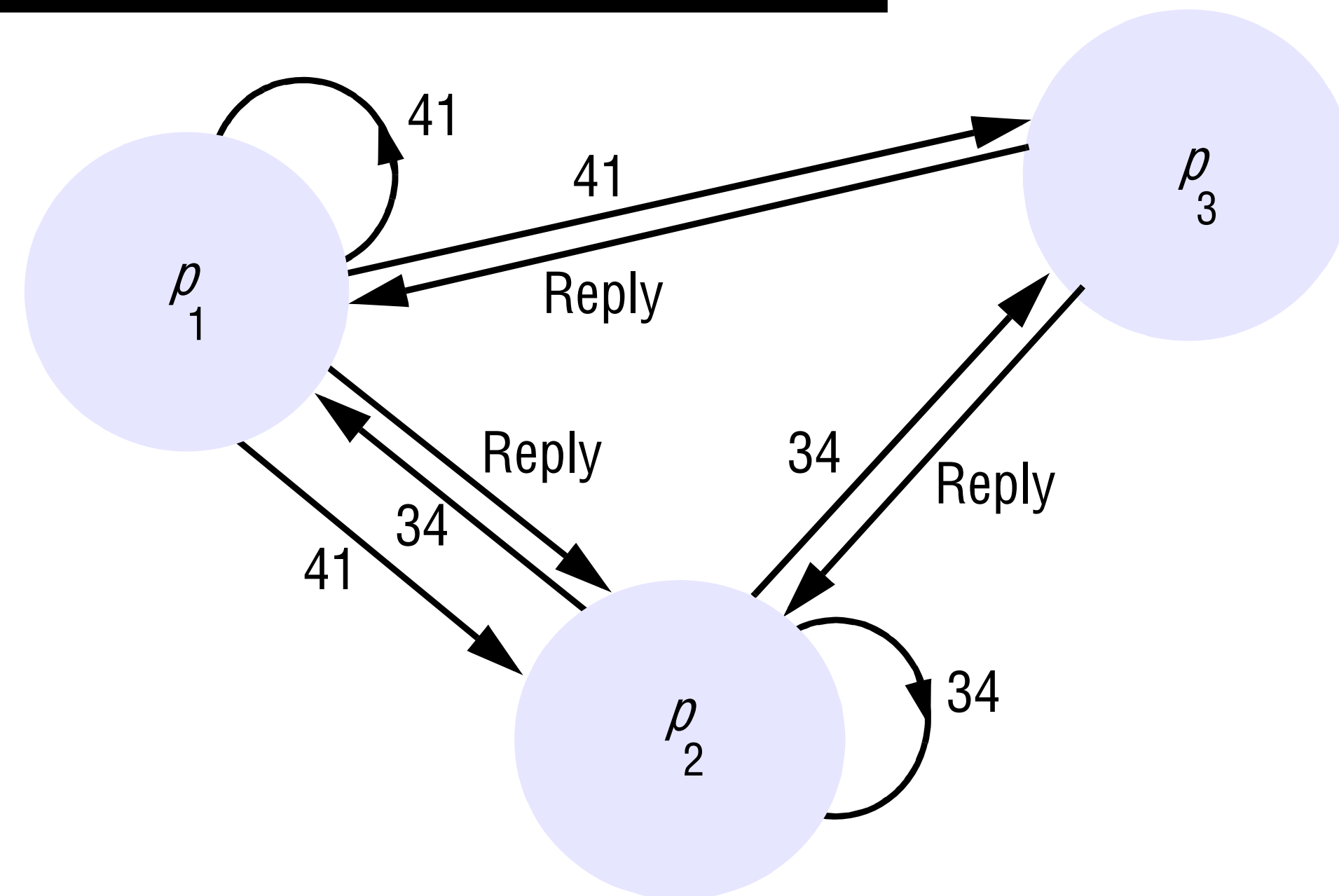


Exclusão Mútua Distribuída

Algoritmo baseado Multicast e Clocks Lógicos

E para que
servem os
relógios
lógicos?

Se dois ou mais processos
requisitam a entrada na região
crítica ao mesmo tempo, aquele
com o menor timestamp
conseguirá N - 1 respostas e entrará
na região crítica



Exclusão Mútua Distribuída

Algoritmo baseado Multicast e Clocks Lógicos

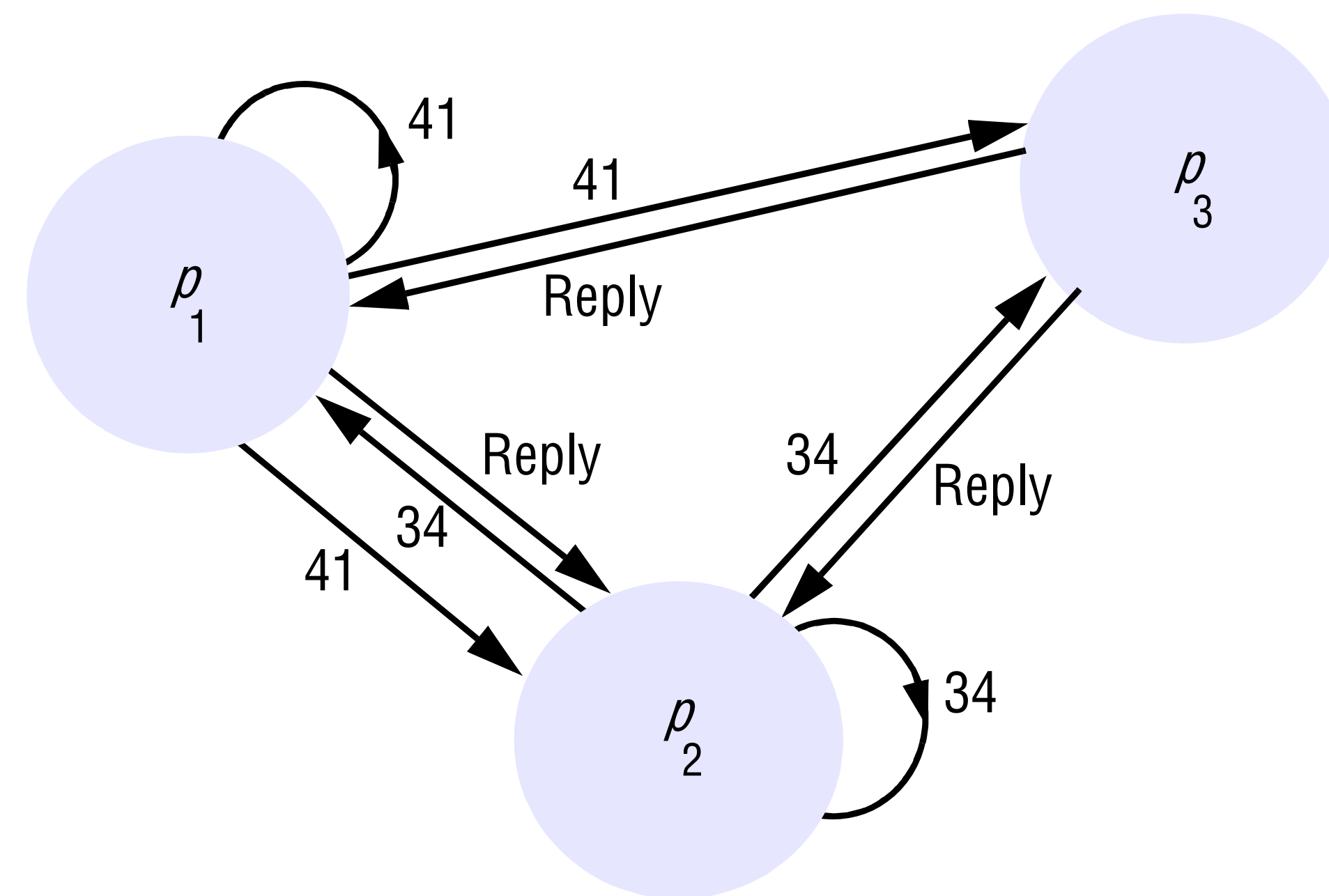
Case

p1 e p2 requisitam entrada
concorrentemente

Timestamp do p1 é 41 e o do p2 é 34

Quando p3 recebe a requisição, ele
retorna a resposta de forma imediata,
pois não quer entrar na região crítica

p1 verifica que a requisição de p2
possui um timestamp menor e responde
para ele



Exclusão Mútua Distribuída

Tolerância a falhas	
Problemas	O que fazer?
O que acontece se mensagens forem perdidas?	Alteração dos algoritmos
O que acontecem se processos sofrerem crashes?	Detector de falhas

Eleições

Escolher um processo único para exercer uma atividade em particular

Case

Poderíamos eleger um servidor no primeiro algoritmo de servidor centralizado para exclusão mútua

É essencial que todos os processos concordem com a escolha da eleição

Um ou mais processos podem pedir por uma chamada a eleições

Eleição

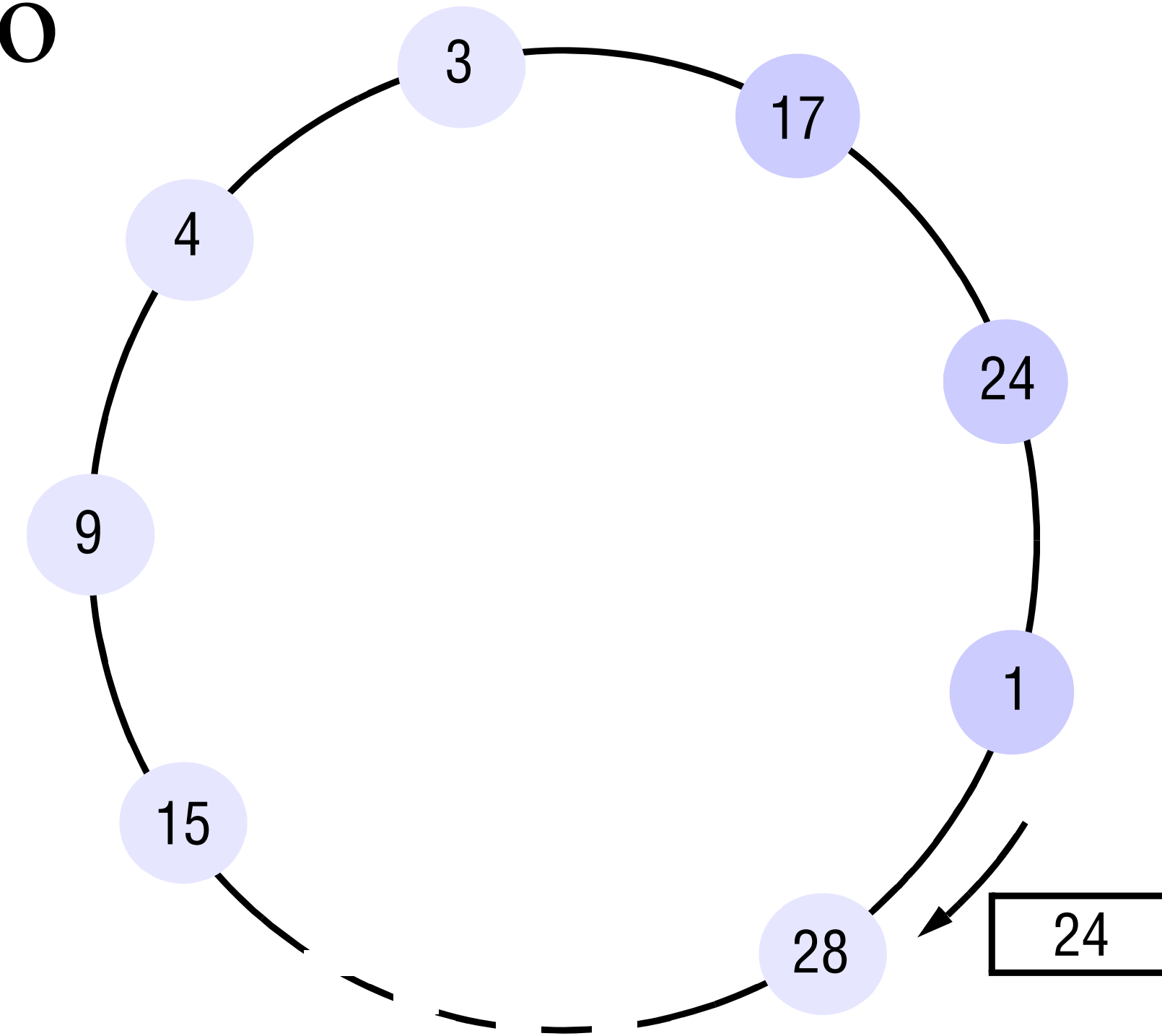
Algoritmo de Eleição Baseado em Anel

Primeiramente, todos os processos são marcados como não participantes da eleição. Qualquer processo pode começar a eleição e seta seu status como participante. Ele coloca o seu identificador em uma mensagem e a envia no sentido do anel.

Quando um processo recebe a requisição de eleição, ele compara o identificador com o seu próprio. Se o recebido é maior, ele repassa a requisição para o próximo vizinho.

Se o identificador é menor e eu sou um não participante, eu substituo o identificador pelo meu próprio e repasso a mensagem.

Se o identificador é menor e eu sou um participante, eu não repasso a mensagem

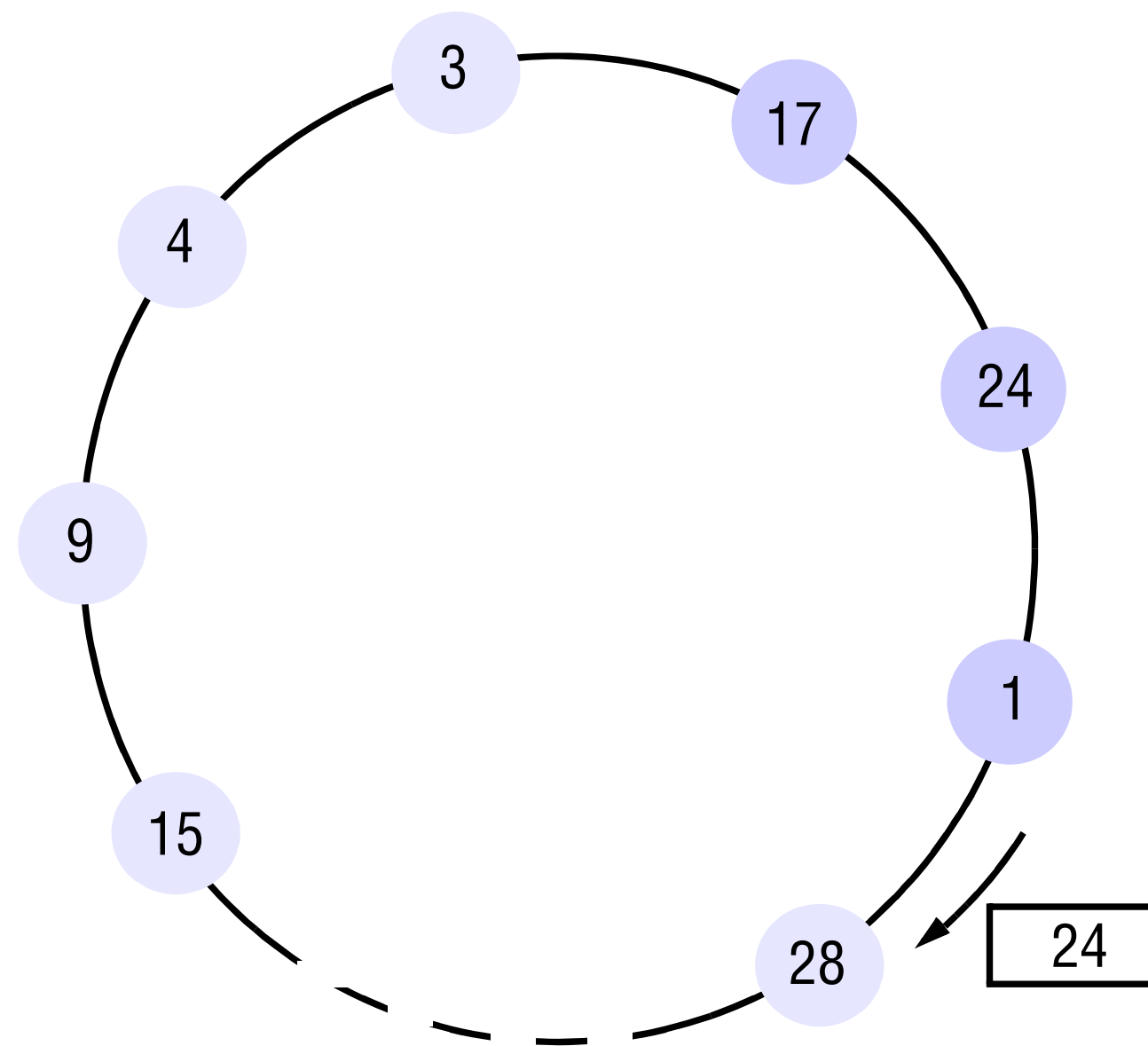


Eleição

Algoritmo de Eleição Baseado em Anel

Se o identificador recebido é o meu próprio, ele é o maior e está eleito o coordenador

O coordenador se marca como não participante e envia uma mensagem para todos os demais avisando o novo líder



Eleição

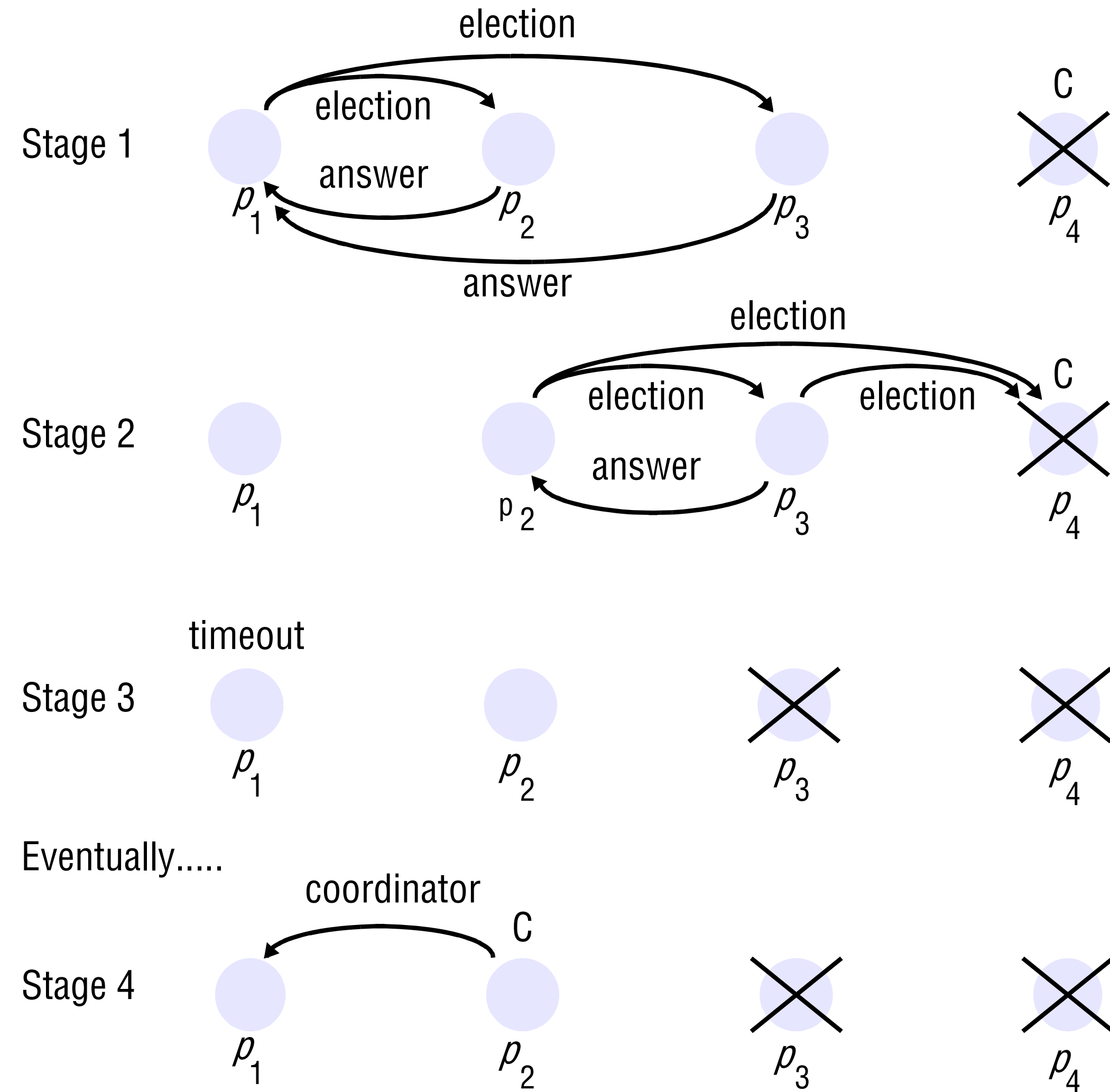
Algoritmo Bully

Processos podem sofrer um *crash* durante a eleição, mas mensagens são ditas sempre confiáveis

Existe timeouts para detectar falhas

Cada processo conhece os demais que possuem identificadores maiores que o seu

Três tipos de perguntas: eleição, pergunta e coordenador

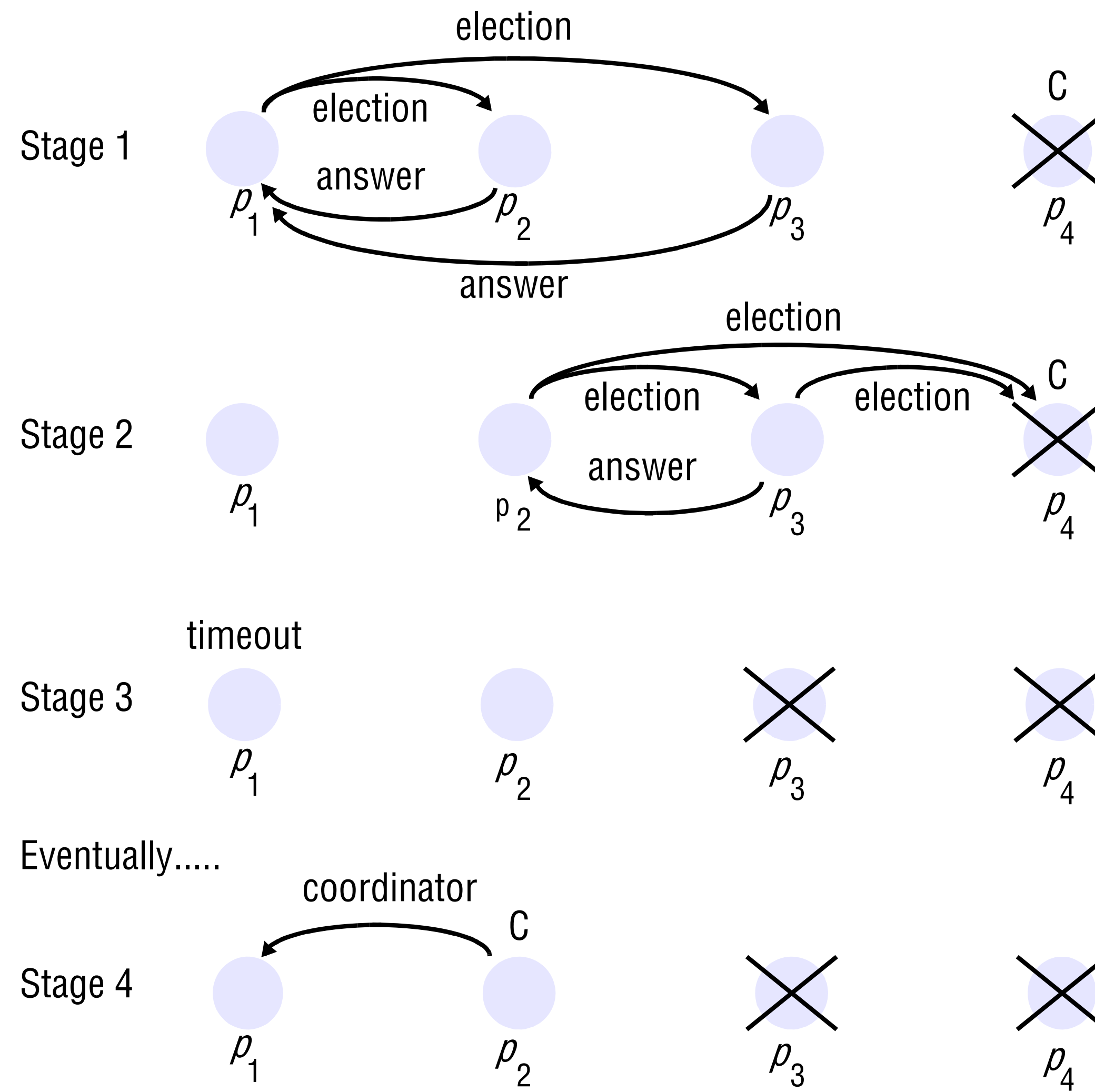


Eleição

Algoritmo Bully

Processo que sabe que tem o maior identificador, simplesmente manda uma mensagem de coordenador para os demais

Processos com identificadores menores podem começar uma eleição com uma mensagem de eleição para processos que possuem identificador maior que o seu



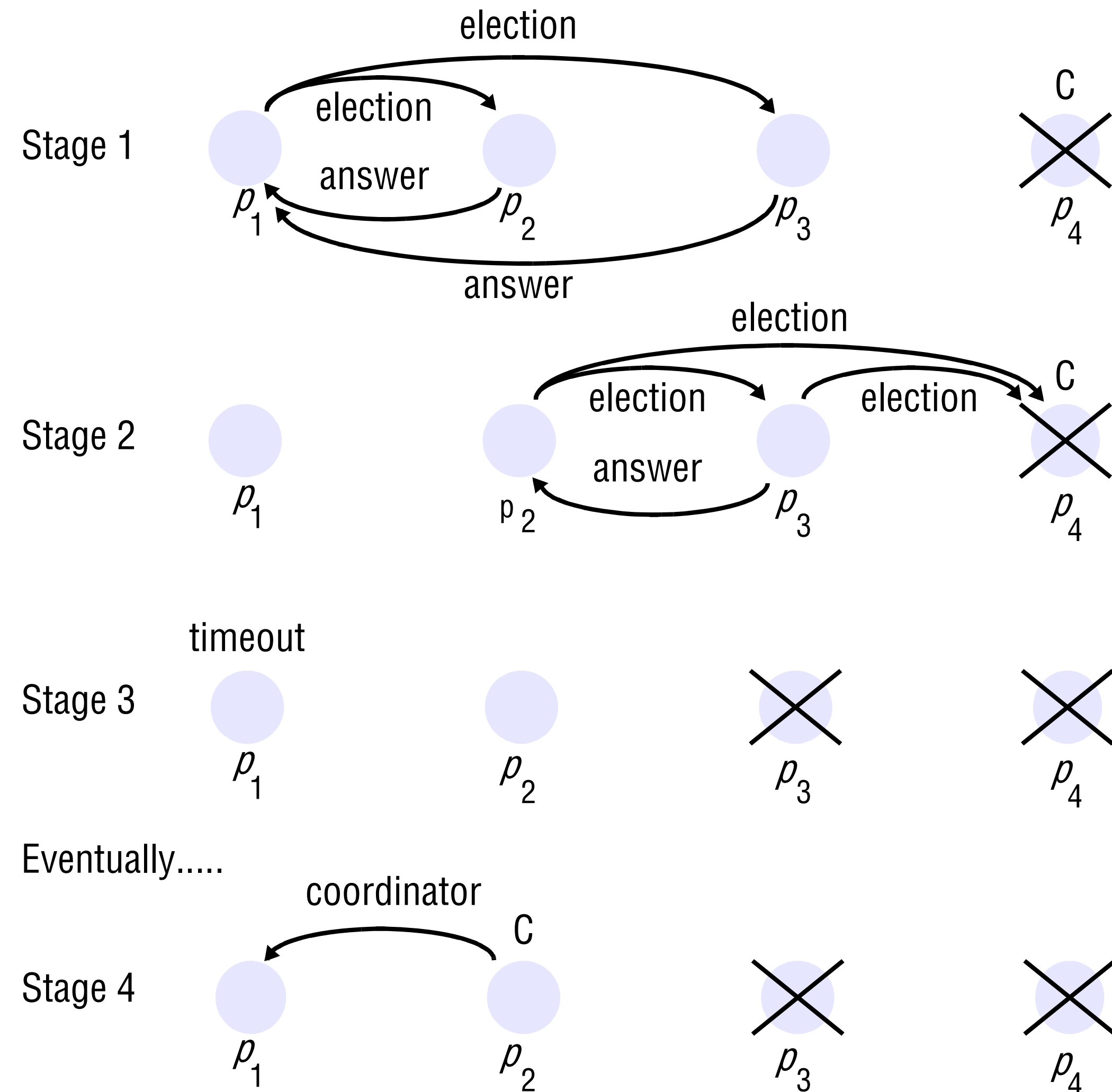
Eleição

Algoritmo Bully

Se nenhuma resposta chega dentro de um tempo T, o processo se considera coordenador e avisa os demais

Se um resposta chegar, ele deve esperar depois pelo anúncio de um novo coordenador

Se um processo recebe uma mensagem de eleição, ele envia um retorno e faz uma nova eleição



Eleição

Algoritmo do tirano (*bully algorithm*):

- Garcia-Molina (1982)
- O algoritmo de Bully serve para eleger um líder (processo coordenador) em algoritmos distribuídos
- Processos são identificados por um identificador numérico, único, fixo e atribuído antes do início da eleição. Cada processo sabe os números de identificação de cada um dos demais processos. Os processos não sabem se quais processos estão ativos e quais estão inativos.
- A topologia não é limitada a um anel e cada um dos processos pode se comunicar com qualquer outro no sistema.
- A execução do algoritmo busca eleger o processo de maior identificador e fazer com que todos reconheçam o novo líder.

Eleição

Bully algorithm

- Desenvolvido por Garcia-Molina em 1982
- Assume que um processo sabe a prioridade de todos outros processos no sistema

Algoritmo

- Quando um processo P_i detecta que o coordenador não está respondendo a um pedido de serviço, ele inicia uma eleição da seguinte forma:
 - a)** P_i envia uma msg ELEIÇÃO para todos processos com prioridade maior que a sua
 - b)** se nenhum processo responde
 - P_i vence a eleição e torna-se coordenador
 - // significa que não há processos ativos com maior prioridade que a sua
 - P_i manda uma msg COORDENADOR para os processos de menor prioridade informando que é o coordenador, deste momento em diante, ●
 - se processo P_j com prioridade maior que P_i responde (msg *ALIVE*)
 - P_i não faz mais nada, P_j assume o controle
 - P_j age como P_i nos passos **a)** e **b)**

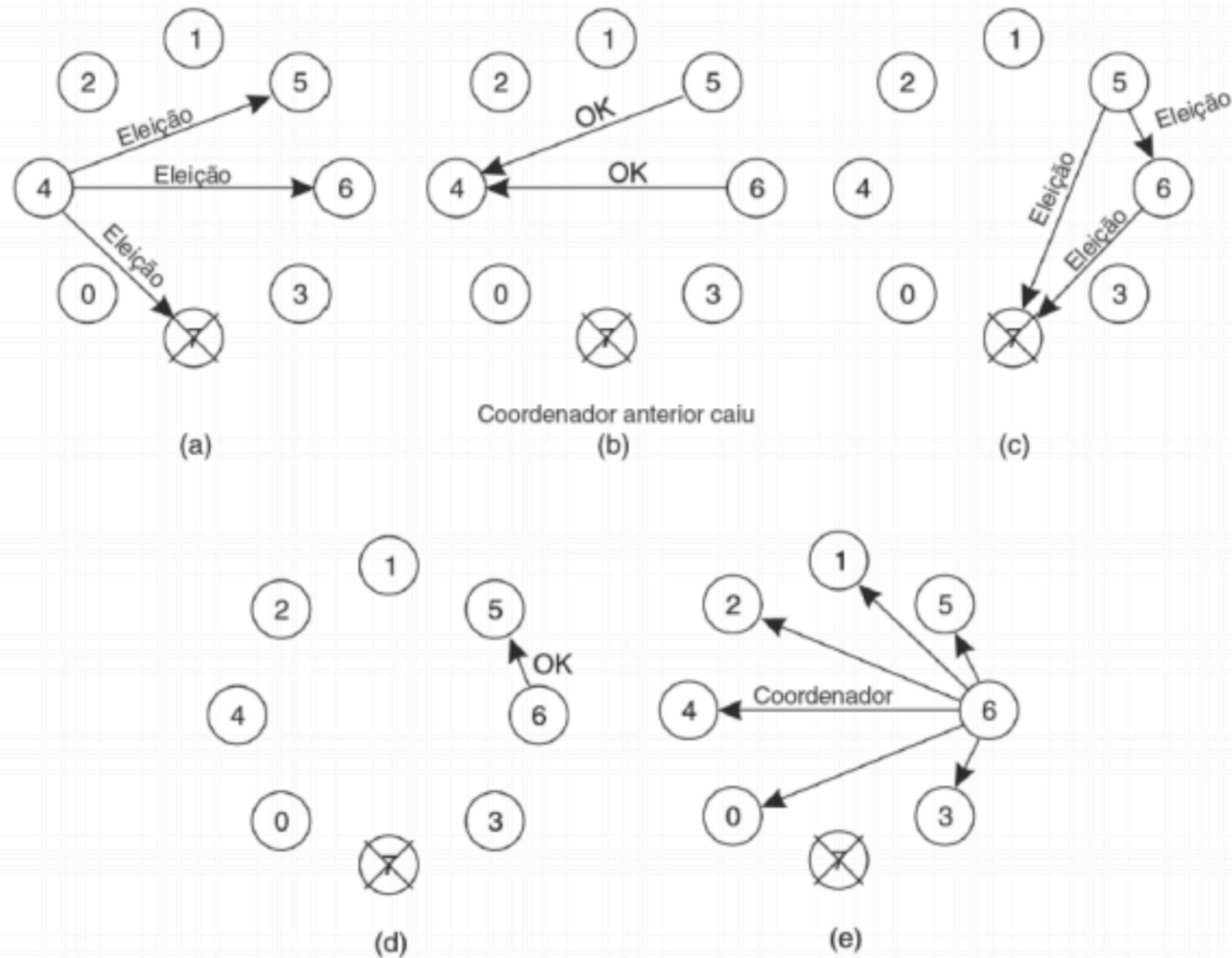


Figura 6.19 Algoritmo de eleição do valentão. (a) O processo 4 convoca uma eleição. (b) Os processos 5 e 6 respondem e mandam 4 parar. (c) Agora, cada um, 5 e 6, convoca uma eleição. (d) O processo 6 manda 5 parar. (e) O processo 6 vence e informa a todos.